

Reviewer Pack — Minimal Rank of p-Adic Differentials over Boolean Tensor Pro...

Entry 9e388a9f766d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 08:40:49 UTC

Minimal Rank of p-Adic Differentials over Boolean Tensor Product Valuations

Entry ID: 9e388a9f766d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 08:40:49 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Differentials) **Field B** (complexity object): Communication Complexity (Boolean Tensor Product)

Statement:

[‘For a given n-bit boolean function f , let $\rho(f)$ be the minimal rank of its p-adic differential form over the finite field with p elements. Then, the boolean tensor product valuation of f is upper-bounded by $O(\rho(f)^2)$.’, ‘Equivalently, for any boolean matrix $M \in \{0,1\}^{N \times N}$, the communication complexity $CC_R(M)$ is upper-bounded by $O(\text{rank}(H_M)^2)$, where H_M is the Hermitian form corresponding to the p-adic differential form of f .’, ‘For all boolean functions f with $n \leq 40$, if $\rho(f) = k$, then $CC_R(f) \leq k^2$.’]

Rationale (proposer’s reasoning):

[‘p-adic differentials have been studied in number theory and arithmetic geometry, but their application to communication complexity is novel. The conjecture aims to bridge these fields by proposing a connection between the rank of p-adic differential forms and the communication complexity of boolean functions.’, ‘This relationship may expose new structures in communication complexity that are not apparent through standard tools like linear algebra or information theory.’, ‘A constructive mapping can be established by computing the p-adic derivative of the characteristic function of f , which corresponds to its p-adic differential form.’]

Taxonomy category: p-adic_differentials (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c5f0e9781640e954

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all boolean functions f with $n \leq 40$, $\rho(f) = k$ and $CC_R(f) \leq k^2$ are true for at least 25 out of 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        pivot_row = i
        for j in range(i + 1, rows):
            if abs(matrix[j][i]) > abs(matrix[pivot_row][i]):
                pivot_row = j
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        if matrix[i][i] == 0:
            continue
        for j in range(i + 1, rows):
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(cols):
                matrix[j][k] += factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

def rank(matrix):
    return gaussian_elimination(matrix)

def boolean_function_to_matrix(f, n):
    M = []
    for i in range(2**n):
        row = [int(x) for x in format(i, f'0{n}b')]
        M.append(row)
    return M

def p_adic_differential_form(M):
    N = len(M)
```

```

H_M = [[0] * N for _ in range(N)]
for i in range(N):
    for j in range(N):
        if i != j:
            diff = sum((M[i][k] - M[j][k]) * (1 << k) for k in range(len(M)))
            H_M[i][j] = diff
return H_M

def communication_complexity(H_M):
    rank_H_M = rank(H_M)
    return rank_H_M ** 2

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = lambda x: random.choice([0, 1])
    M = boolean_function_to_matrix(f, n)
    H_M = p_adic_differential_form(M)
    rho_f = rank(H_M)
    CC_R_f = communication_complexity(H_M)
    conjecture_holds = CC_R_f <= rho_f ** 2
    counterexample = "" if conjecture_holds else f"CC_R(f)={CC_R_f}, rho(f)^2={rho_f**2}"
    return {
        "metric_name": "communication_complexity",
        "metric_value": CC_R_f,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = result["counterexample"]
                first_failing_seed = seed
                break
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete its intended task of verifying the conjecture for all boolean functions f with $n \leq 40$. | next: Re-run the test with a longer timeout or increase system resources to ensure it can complete the computation.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11175
2	propose	ollama_remote	glm4:latest	0	0	10112
3	propose	ollama_remote	glm4:latest	0	0	12093
4	preregistration	ollama_remote	glm4:latest	0	0	5673
5	novelty	ollama_remote	glm4:latest	0	0	5015
6	novelty	ollama_remote	glm4:latest	0	0	5185
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16877
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13705
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10895
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11391
11	judge	ollama_remote	glm4:latest	0	0	58111

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160232 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/9e388a9f766d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9e388a9f766d.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9e388a9f766d.tar.gz` (if generated)